

SD VOC - Using and Managing the VOC

How the command processor reads the VOC, the verbs that manage it, and what account creation puts there.

This page continues SD VOC - Structure and Usage.

How the command processor uses the VOC

The command processor is a BASIC program, `$CPROC`, and it runs a loop: show any waiting messages, print the prompt, read a line, dispatch it, repeat. The first word of the line is looked up in the account's `voc` file.

NAME RESOLUTION

A name is tried **as typed, then lower case, then upper case**. This is the change that lets a lower-case `voc`, `bp` and `newvoc` on disk coexist with code and habits that still type names in upper case.

Account names are the deliberate exception. They are still folded upward, which is what makes signing in case-insensitive.

IF THE VOC HAS NOTHING, THE CATALOGUE IS TRIED

A name that is not in the VOC is looked for in the **private catalogue** of the account and then in the **global catalogue**, `gcat`. A hit is treated exactly as though the VOC had held `V / CA / name`. This is why a program you have catalogued can be run by typing its name with no VOC record of its own.

If neither has it, the command is refused by name.

Verbs that manage the VOC

SET.FILE — WRITE A Q-POINTER

```
set.file account file.name pointer.name
```

Writes a Q-pointer into the VOC — field 1 `Q`, field 2 the account name, field 3 the file name. After it, *pointer.name* is usable wherever a file name is, and the data stays where it is.

The account must be in the accounts register. The name is folded to upper case before the register is read. If it is not there you get *Account name '...' is not in register* and nothing is written.

UPDATE.ACCOUNTS — REFRESH FROM NEWVOC

```
update.accounts
```

Re-runs `LOGIN`'s `update.voc` paragraph, which copies any changed records from the system `NEWVOC` into the account's VOC. **Since 17 Aug 2026 this is incremental** — it re-copies only the records whose stamp has changed, not the whole file. `TIER.OMIT.STANDARD` and `TIER.ADD.ADMINISTRATOR` in `NEWVOC` control what each account type receives.

The tier test is the one this record exists for. `update.accounts` on a standard account must not give back the verbs that were withheld when the account was created. A standard account starts with fewer verbs than an administrator account, and `update.accounts` preserves that difference.

COPY — COPY RECORDS INTO THE VOC

```
copy from voc messages,myname
copy from src.file to dict tgt.file record.id
```

`copy` can write into the VOC the way it writes into any file. The `from voc` form copies a VOC record under a new name, which is how an alias is made without `alias`.

DELETE — REMOVE A VOC RECORD

```
delete voc name
```

Removes the record. `.d` **name at the command prompt asks first**; `delete voc name` does not.

.S AND .R — SAVE AND RECALL

```
.s name s e
.r name
```

`.s` writes stack lines *s* to *e* into the VOC as *name* — a sentence when the range is one line, a paragraph when it is more. `.r` loads a sentence or paragraph from the VOC into the stack. **The record type follows from what you saved** rather than from anything you say.

THE LIST FAMILY

Ten verbs list VOC entries by type. They are all R-records into `SD.VOCLIB`:

VERB	LISTS
<code>listf</code>	all files (F-records)

VERB	LISTS
<code>listfl</code>	local files only
<code>listfr</code>	remote files only
<code>listk</code>	all keywords (K-records)
<code>listpa</code>	all paragraphs (PA-records)
<code>listph</code>	all phrases (PH-records)
<code>listq</code>	all Q-pointers
<code>listr</code>	all remote references (R-records)
<code>lists</code>	all sentences (S-records)
<code>listv</code>	all verbs (V-records)

COUNT — COUNT THE RECORDS

```
count voc
count voc with v = "V"
```

`count` over the VOC gives the total record count. With a selection clause it counts by type or by any other field.

CT — DISPLAY A RECORD

```
ct voc name
```

Displays the record field by field. This is the way to see what a VOC entry actually contains without an editor.

The VOC and account creation

`CREATE.ACCOUNT` copies the VOC from `voc_template` in the system directory. The template holds 426 records; `NEWVOC` holds 395. The difference is that `voc_template` carries the A-verbs — `abort`, `alias`, `all`, `after`, `and` and the rest — that are also keywords, and `NEWVOC` does not. An account gets the union of the two, less whatever its tier omits.

TIER	WHAT IT GETS
administrator	everything in <code>voc_template</code> plus everything in <code>NEWVOC</code> , less <code>TIER.OMIT.STANDARD</code>

TIER	WHAT IT GETS
standard	everything in <code>voc_template</code> plus everything in <code>NEWVOC</code> , less <code>TIER.OMIT.STANDARD</code> (42 names)
programmer	standard plus the programmer verbs from <code>TIER.ADD.ADMINISTRATOR</code>

The tier is in the VOC, not in the verb. A standard account does not have `create.file` because the VOC record for it is not there, not because a security subroutine refuses it. The name is simply not recognised. This is the design: tiering is done by giving or withholding the record.

Case on disk

Since 18 Aug 2026 the VOC ids are stored in lower case — `list`, `create.account` and so on. This changes nothing about what you type — SD tries a name as typed, then lower, then upper. What it changes is what SD prints back: `CT VOC LIST` answers `VOC list`.

The file-pointer entries — `bp`, `bp.out`, `gpl.bp`, `gpl.bp.out` — moved to lower case on 19 Aug 2026. `VOC`, `NEWVOC`, `ACCOUNTS`, `MESSAGES`, `SYSCOM` and `QFILE` are still upper case on disk and are next.

What is not here

There is no verb that edits the VOC directly. `ED` edits any file, and `ED VOC name` is how you edit a VOC record by hand. But there is no verb whose purpose is to create or modify VOC entries — `set.file` writes one kind, `.s` writes two, and everything else is done with `ED` or with `copy from voc`.

PROC is removed. A `PQ`-type record is refused by name. Nothing that ships is type `PQ`, so this can only be met in a VOC record somebody wrote.

menu records are dispatched but no menu ships. Type `M` is handled and there is no shipped example to look at.

EDIT is in the VOC but cannot run. Its type field reads `Verb - Full screen editor`, which is not `V`, so the command processor does not dispatch it. The record is there so that `EDIT` at the prompt prints *"Full screen editor is no longer supported"* rather than *"verb not found"*.

See also

SD TCL - The Command Processor · SD TCL - Files and Records · SD Basic - File Handling · SD Basic - Dynamic Arrays.

